

ATNX Phase 2 — Google Trends Virality Score, Sorting & Charts

Overview

Build on the Phase 1 Chrome extension + web app. When a new capture arrives with AI analysis, automatically query Google Trends for the identified subject, calculate a virality score, sort the capture list by that score, and display an inline trend chart next to each entry.

Architecture Changes

Phase 1 flow (existing):

Extension → AI Analysis → POST to /api/captures → Display in list

Phase 2 additions:

POST to /api/captures

|

▼

Extract search term from AI analysis (name field)

|

▼

Query Google Trends API for that term

|

▼

Calculate virality score from trends data

|

▼

Store capture + trends data + score

|

▼

Display list sorted by virality score
with inline sparkline/chart per entry

Tech Additions

- **google-trends-api:** npm package for querying Google Trends (`npm install google-trends-api`)
- **recharts:** For rendering trend charts in the dashboard (`npm install recharts`)
- Everything else stays the same from Phase 1

Part 1: Google Trends Integration

Install Dependency

```
bash  
  
cd atnx-web  
npm install google-trends-api
```

Create Trends Service — lib/trends.ts

This module handles all Google Trends queries and score calculation.

```
typescript
```

```
// lib/trends.ts

import googleTrends from 'google-trends-api';

interface TrendsDataPoint {
  date: string;      // ISO date string
  value: number;     // 0-100 Google Trends interest value
}

interface TrendsResult {
  term: string;
  dataPoints: TrendsDataPoint[];
  viralityScore: number;
  peakValue: number;
  currentValue: number;
  trend: 'rising' | 'falling' | 'stable' | 'spiking' | 'new';
  fetchedAt: string;
}

export async function fetchTrendsData(searchTerm: string): Promise<TrendsResult> {
  try {
    // Query interest over time – last 7 days, hourly granularity
    const results = await googleTrends.interestOverTime({
      keyword: searchTerm,
      startTime: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000), // 7 days ago
      endTime: new Date(),
      granularTimeResolution: true, // Hourly data when available
    });

    const parsed = JSON.parse(results);
    const timelineData = parsed.default?.timelineData || [];

    const dataPoints: TrendsDataPoint[] = timelineData.map((point: any) => ({
      date: new Date(parseInt(point.time) * 1000).toISOString(),
      value: point.value?.[0] ?? 0,
    }));

    // Calculate virality score
    const viralityScore = calculateViralityScore(dataPoints);
    const values = dataPoints.map(d => d.value);
    const currentValue = values.length > 0 ? values[values.length - 1] : 0;
    const peakValue = values.length > 0 ? Math.max(...values) : 0;
    const trend = determineTrend(dataPoints);
```

```

    return {
      term: searchTerm,
      dataPoints,
      viralityScore,
      peakValue,
      currentValue,
      trend,
      fetchedAt: new Date().toISOString(),
    };
  } catch (error) {
    console.error(`Google Trends query failed for "${searchTerm}":`, error);

    // Return a zero-score result on failure – don't break the pipeline
    return {
      term: searchTerm,
      dataPoints: [],
      viralityScore: 0,
      peakValue: 0,
      currentValue: 0,
      trend: 'new',
      fetchedAt: new Date().toISOString(),
    };
  }
}

```

Virality Score Calculation — lib/trends.ts (continued)

The virality score is a composite number (0-1000 scale) derived from Google Trends data. This is the beginning of the proprietary ATNX Virality Index.

typescript

```

function calculateViralityScore(dataPoints: TrendsDataPoint[]): number {
  if (dataPoints.length === 0) return 0;

  const values = dataPoints.map(d => d.value);
  const len = values.length;

  // --- Component 1: Current Interest (0-100) ---
  // How popular is it RIGHT NOW
  const currentValue = values[len - 1] || 0;

  // --- Component 2: Momentum (0-100) ---
  // Is it growing or declining? Compare recent vs older data
  // Split the data into two halves and compare averages
  const midpoint = Math.floor(len / 2);
  const olderHalf = values.slice(0, midpoint);
  const recentHalf = values.slice(midpoint);

  const olderAvg = olderHalf.length > 0
    ? olderHalf.reduce((a, b) => a + b, 0) / olderHalf.length
    : 0;
  const recentAvg = recentHalf.length > 0
    ? recentHalf.reduce((a, b) => a + b, 0) / recentHalf.length
    : 0;

  // Momentum: positive means growing, negative means declining
  // Normalize to 0-100 range where 50 = stable
  let momentum = 50;
  if (olderAvg > 0) {
    const changeRatio = (recentAvg - olderAvg) / olderAvg;
    momentum = Math.max(0, Math.min(100, 50 + (changeRatio * 100)));
  } else if (recentAvg > 0) {
    momentum = 100; // Went from nothing to something – max momentum
  }

  // --- Component 3: Spike Detection (0-100) ---
  // Is there a sudden sharp increase? Look at the last few data points
  const recentPoints = values.slice(-6); // Last ~6 hours or data points
  const priorPoints = values.slice(-12, -6);

  const recentMax = recentPoints.length > 0 ? Math.max(...recentPoints) : 0;
  const priorMax = priorPoints.length > 0 ? Math.max(...priorPoints) : 0;

  let spike = 0;

```

```

if (priorMax > 0) {
  spike = Math.min(100, ((recentMax - priorMax) / priorMax) * 100);
} else if (recentMax > 20) {
  spike = 80; // Appeared from nothing with decent volume
}
spike = Math.max(0, spike);

// --- Component 4: Consistency (0-100) ---
// Is the interest sustained or just a blip?
// Count how many data points are above 10% of peak
const peak = Math.max(...values, 1);
const threshold = peak * 0.1;
const aboveThreshold = values.filter(v => v >= threshold).length;
const consistency = Math.min(100, (aboveThreshold / len) * 100);

// --- Composite Score ---
// Weighted combination, scaled to 0-1000
const composite = (
  currentValue * 0.35 + // Current interest matters most
  momentum * 0.30 + // Growth trajectory
  spike * 0.20 + // Spike detection (is it going viral RIGHT NOW)
  consistency * 0.15 // Sustained attention
);

// Scale from 0-100 composite to 0-1000 index
return Math.round(composite * 10);
}

function determineTrend(dataPoints: TrendsDataPoint[]): 'rising' | 'falling' | 'stable' | 'new' {
  if (dataPoints.length < 3) return 'new';

  const values = dataPoints.map(d => d.value);
  const recent = values.slice(-6);
  const prior = values.slice(-12, -6);

  const recentAvg = recent.reduce((a, b) => a + b, 0) / recent.length;
  const priorAvg = prior.length > 0
    ? prior.reduce((a, b) => a + b, 0) / prior.length
    : 0;

  if (priorAvg === 0 && recentAvg > 0) return 'new';

  const changePercent = priorAvg > 0 ? ((recentAvg - priorAvg) / priorAvg) * 100 : 0;

```

```
if (changePercent > 100) return 'spiking';
if (changePercent > 15) return 'rising';
if (changePercent < -15) return 'falling';
return 'stable';
}
```

Score Breakdown

The virality score (0-1000) is composed of:

Component	Weight	What it measures
Current	35%	Raw interest level right now
Momentum	30%	Growth trajectory (recent vs older data)
Spike	20%	Sudden sharp increases in the last few hours
Consistency	15%	Sustained interest vs one-off blip

This weighting can be tuned later. For MVP these weights favor things that are actively going viral (high current + high momentum + spike detected) over things that are consistently popular but not trending (high consistency but low momentum).

Part 2: Update API Route

/api/captures/route.ts — Updated

When a new capture is POSTed, query Google Trends before storing:

```
typescript
```

```
import { fetchTrendsData } from '@lib/trends';

let captures: any[] = [];

export async function POST(request: Request) {
  const data = await request.json();

  // Extract search term from AI analysis
  // Use the "name" field from the AI response as the search query
  const searchTerm = data.analysis?.name || '';

  // Query Google Trends
  let trendsData = null;
  if (searchTerm && searchTerm.length > 1) {
    trendsData = await fetchTrendsData(searchTerm);
  }

  const capture = {
    id: data.id || Date.now().toString(),
    timestamp: data.timestamp || new Date().toISOString(),
    pageUrl: data.pageUrl,
    pageTitle: data.pageTitle,
    screenshot: data.screenshot,
    analysis: data.analysis,
    trends: trendsData,
    viralityScore: trendsData?.viralityScore ?? 0,
  };

  captures.unshift(capture);

  // Re-sort all captures by virality score (highest first)
  captures.sort((a, b) => b.viralityScore - a.viralityScore);

  return Response.json({ success: true, id: capture.id, viralityScore: capture.viralityScore });
}

export async function GET() {
  // Already sorted by viralityScore (highest first)
  return Response.json({ captures });
}
```

Rate Limiting Note

Google Trends is an unofficial API and can rate-limit aggressive querying. Add a simple delay/cache:

```
typescript

// lib/trends-cache.ts

const cache = new Map<string, { data: any; expiry: number }>();
const CACHE_TTL = 5 * 60 * 1000; // 5 minute cache

export function getCached(term: string) {
  const entry = cache.get(term.toLowerCase());
  if (entry && Date.now() < entry.expiry) return entry.data;
  return null;
}

export function setCache(term: string, data: any) {
  cache.set(term.toLowerCase(), { data, expiry: Date.now() + CACHE_TTL });
}
```

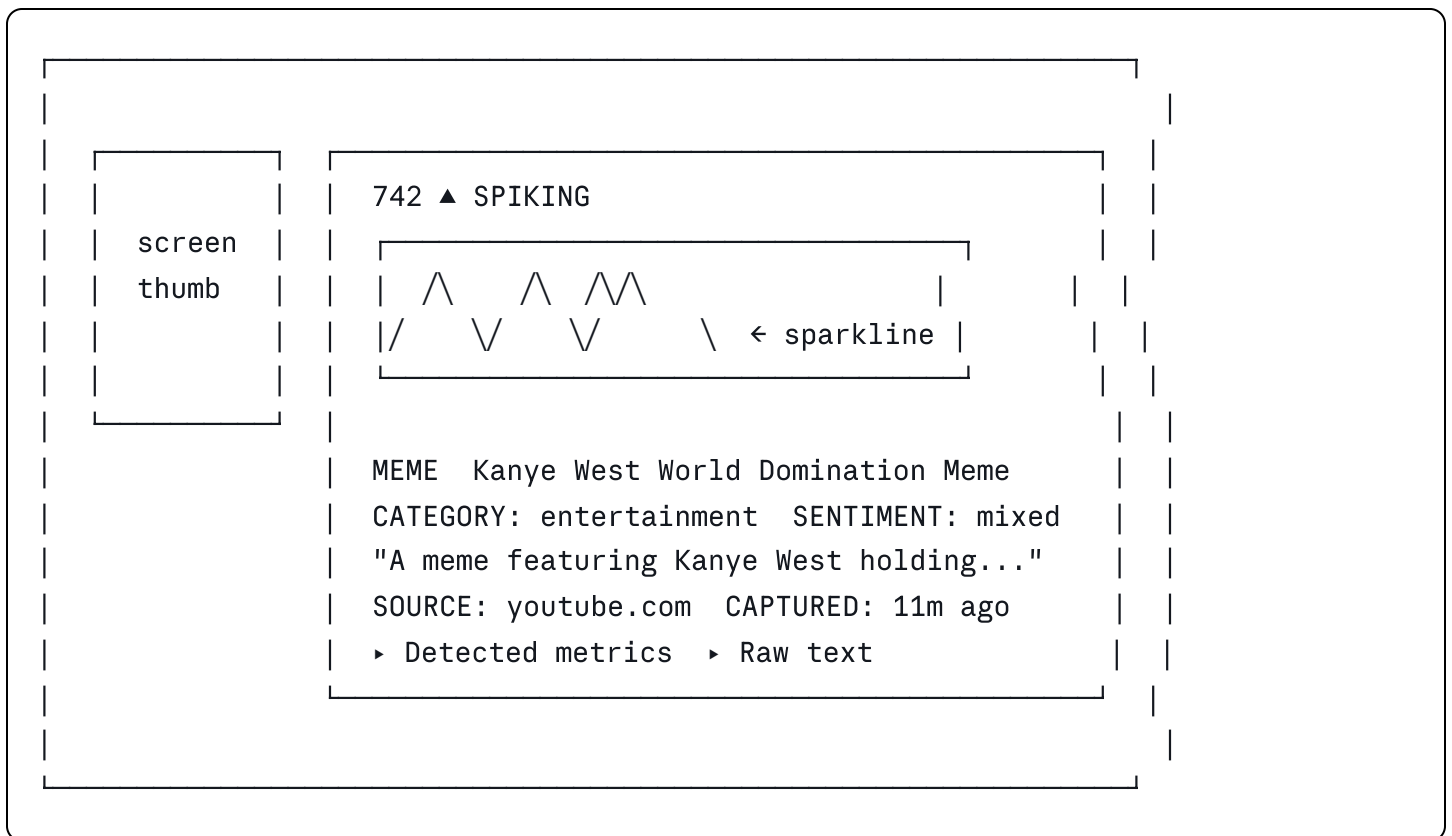
Use this cache in `fetchTrendsData` — check cache first, only query Google if cache miss.

Part 3: Update Dashboard UI

Capture Card — Updated Layout

Each capture card now shows:

1. **Virality score badge** — large, prominent number (0-1000) with color coding
2. **Trend indicator** — arrow icon showing rising/falling/spiking/stable
3. **Sparkline chart** — small inline chart showing the Google Trends data
4. **All existing Phase 1 data** — thumbnail, type, name, category, sentiment, etc.



Virality Score Badge

Color code the score:

typescript

```
function getScoreColor(score: number): string {  
  if (score >= 800) return '#FF0040';    // Red hot – extremely viral  
  if (score >= 600) return '#FF6600';    // Orange – very high interest  
  if (score >= 400) return '#FFD700';    // Gold – trending  
  if (score >= 200) return '#00FF66';    // Green – moderate interest  
  return '#888888';                      // Gray – low/no interest  
}
```

Display the score prominently at the top-right of each card. Large font, monospace, with the colored background.

Trend Indicator

Show alongside the score:

```
▲ SPIKING    - score >= 800 or trend === 'spiking'
➤ RISING     - trend === 'rising'
➤ STABLE     - trend === 'stable'
➤ FALLING    - trend === 'falling'
★ NEW        - trend === 'new'
```

Sparkline Chart Component

Use `recharts` to render a small inline chart for each capture showing the Google Trends data over the past 7 days.

tsx

```

// components/TrendSparkline.tsx

'use client';

import { LineChart, Line, ResponsiveContainer, YAxis } from 'recharts';

interface SparklineProps {
  dataPoints: { date: string; value: number }[];
  color?: string;
  width?: number;
  height?: number;
}

export function TrendSparkline({
  dataPoints,
  color = '#00FF66',
  width = 280,
  height = 60
}: SparklineProps) {
  if (!dataPoints || dataPoints.length === 0) {
    return (
      <div
        style={{ width, height }}
        className="flex items-center justify-center text-atnx-text-muted text-xs font
      >
        No trend data
      </div>
    );
  }

  return (
    <div style={{ width, height }}>
      <ResponsiveContainer width="100%" height="100%">
        <LineChart data={dataPoints}>
          <YAxis domain={[0, 100]} hide />
          <Line
            type="monotone"
            dataKey="value"
            stroke={color}
            strokeWidth={1.5}
            dot={false}
            isAnimationActive={false}
          />

```

```

        </LineChart>
      </ResponsiveContainer>
    </div>
  );
}

```

Sorting Controls

Add a simple sort control at the top of the dashboard:

```

| ATNX [4] Captures |
| Attention Exchange |
|
| Sort by: [Virality ▼] [Newest] [Category] |
|

```

Three sort modes:

- **Virality** (default): Highest virality score first
- **Newest**: Most recent capture first (Phase 1 behavior)
- **Category**: Group by AI-detected category

Implement as simple state toggle on the frontend. The API always returns sorted by virality; the frontend can re-sort client-side for the other modes.

Part 4: Updated Capture Card Component

components/CaptureCard.tsx

```
tsx
```

```
// Structure – implement fully with proper Tailwind styling
```

```
interface CaptureCardProps {
  capture: {
    id: string;
    timestamp: string;
    pageUrl: string;
    pageTitle: string;
    screenshot: string;
    analysis: {
      type: string;
      name: string;
      description: string;
      category: string;
      sentiment: string;
      platforms_detected: string[];
      metrics_detected: Record<string, any>;
      virality_signals: string;
      raw_text: string;
    };
    trends: {
      dataPoints: { date: string; value: number }[];
      viralityScore: number;
      peakValue: number;
      currentValue: number;
      trend: string;
    } | null;
    viralityScore: number;
  };
}
```

```
export function CaptureCard({ capture }: CaptureCardProps) {
  // Render:
  // 1. Screenshot thumbnail (left side)
  // 2. Main content (right side):
  //   a. Virality score badge + trend indicator (top right, prominent)
  //   b. Sparkline chart
  //   c. Type badge + Name
  //   d. Category + Sentiment
  //   e. AI description in quotes
  //   f. Virality signals text from AI
  //   g. Source URL + capture time
}
```

```
// h. Collapsible: detected metrics, raw text
}
```

Part 5: Handle Edge Cases

Search Term Extraction

The AI might return names that don't work well as Google Trends queries. Add a normalization step:

typescript

```
function normalizeSearchTerm(analysis: any): string {
  let term = analysis?.name || '';

  // Remove quotes
  term = term.replace(/["']/g, '');

  // If the name is too specific/long, try to shorten it
  // Google Trends works best with 1-3 word queries
  if (term.split(' ').length > 4) {
    // Take first 3 meaningful words, skip articles
    const stopWords = ['the', 'a', 'an', 'of', 'and', 'in', 'on', 'at', 'for', 'to'];
    const words = term.split(' ').filter(w => !stopWords.includes(w.toLowerCase()));
    term = words.slice(0, 3).join(' ');
  }

  return term.trim();
}
```

Zero Results Handling

If Google Trends returns no data (term too niche, misspelled, etc.):

- Still store the capture
- Show virality score as 0 with "No trend data available" message
- Show the "★ NEW" trend indicator
- Sparkline shows "No data" placeholder
- The entry falls to the bottom of the sorted list

Duplicate/Similar Captures

For now, DO NOT deduplicate. Each capture is its own entry even if it's about the same subject. Phase 3 will handle entity grouping. Just note: if two captures have the same search term, the Google Trends cache will serve the same data for both (which is correct — same subject, same virality).

Implementation Priority

Build in this order:

1. **lib/trends.ts** — Google Trends query + virality score calculation
 2. **lib/trends-cache.ts** — Simple caching layer
 3. **Update /api/captures/route.ts** — Integrate trends query into POST handler, add sorting
 4. **TrendSparkline component** — Inline chart using recharts
 5. **Update CaptureCard** — Add score badge, trend indicator, sparkline
 6. **Sort controls** — Add sort toggle to dashboard header
 7. **Polish** — Loading states for trends query, error handling, edge cases
-

Important Notes

- **google-trends-api is unofficial:** It scrapes Google Trends and may break if Google changes their frontend. This is fine for MVP. A production version would use multiple data sources (Phase 3+).
 - **Trends query adds latency:** The POST to /api/captures will now take 2-5 seconds longer due to the Google Trends query. Show a loading/processing state on the dashboard while this completes. Consider making the trends fetch async — store the capture immediately with score 0, then update it when trends data arrives.
 - **Score recalculation:** Virality scores are snapshots at capture time. They do NOT auto-update in this phase. Each time someone captures the same subject, it gets a fresh score. Auto-refresh is a future enhancement.
 - **The 0-1000 scale:** Is arbitrary but intentional. It gives room for granularity and feels more "index-like" than 0-100. It also maps well to future market pricing (a meme at virality index 742 feels like a price point you'd trade around).
-

Testing

Test these scenarios to verify the pipeline:

1. **Capture a currently trending topic** (check Google Trends beforehand for something at 80-100 interest) — should show high virality score (600+) with an upward sparkline
 2. **Capture an obscure/niche topic** — should show low score (0-100), flat or no sparkline
 3. **Capture two different topics** — list should sort with higher virality score on top
 4. **Capture the same topic twice** — both entries should show similar scores (cached), listed adjacent
 5. **Capture something with a long/complex name** — search term normalization should handle it gracefully
 6. **Capture something that doesn't exist in Google Trends** — should show 0 score with "No data" state, not crash
-

Success Criteria for Phase 2

The build is done when:

1. ☒ Every new capture automatically queries Google Trends for the identified subject
2. ☒ A virality score (0-1000) is calculated from the trends data
3. ☒ The capture list is sorted by virality score (highest first) by default
4. ☒ Each capture card displays the virality score prominently with color coding
5. ☒ Each capture card shows a trend indicator (rising/falling/spiking/stable/new)
6. ☒ Each capture card shows an inline sparkline chart of the Google Trends data
7. ☒ Sort controls allow switching between virality, newest, and category views
8. ☒ Captures with no trends data gracefully show zero state (not errors)
9. ☒ The full pipeline (capture → AI → trends → display) works end to end